S

Sahaara AI

System Architecture & Technical Manual

A Deep-Dive into the Frontend, Voice Mode, 3D WebGL & Clinical Engines

---

PROJECT SPECIFICATION & SYSTEM OVERVIEW

* Application: Sahaara AI (CareTrack Healthcare & Wellbeing Suite)
* Repository: https://github.com/binary-collector/SahaaraAI.git
* Target Platform: Modern Chromium Browsers, PWA, Standalone Static Web
* Architecture: Client-Side Serverless PWA + Optional Streaming LLM Gateway
* Core Engines: SpeechRecognition, WebGL 3D Orb, Clinical F-D-E Engine

DOCUMENT CONTROL & METADATA

Classification: Production Technical Documentation / Engineering Whitepaper
Author: Sahaara AI Core Engineering Team
Generation Date: October 31, 2026
Format: PDF 1.4 Native Portable Document (Zero-Dependency Engine)

Notice: Sahaara AI provides automated self-care guidance and is not a substitute for professional clinical judgment.

1. Executive Summary & Architectural Overview

Sahaara AI is a modern healthcare and patient guidance web application designed to empower patients in understanding and managing daily care plans, prescriptions, vital habits, and mental wellbeing. Built as a high-performance Progressive Web App (PWA), it combines client-side conversational AI, an interactive 3D WebGL voice orb, speech synthesis and recognition, and clinical adherence tracking.

1.1 High-Level Architecture Model

Sahaara AI follows a Client-Side Serverless Architecture with Hybrid Offline Resilience. The system operates entirely client-side when running in standalone mode (via any static host or file server), while maintaining seamless integration with optional external LLM endpoints (e.g., Google Cloud Vertex AI, Google Gemini, OpenAI, or local proxy gateways).

Tier / Layer	Primary Technology	Key Responsibilities
Presentation	HTML5, Vanilla ES6+, CSS3	Reactive UI rendering, modal s
Conversational AI	Sahaara Intelligence Engine	Dual-route intent matcher, saf
Voice AI & 3D	Three.js WebGL, Web Speech	Audio-reactive 3D orb, STT rec
Clinical State	Local State Store & Math Lib	Care plan scheduling, 7-day ad
Data Persistence	IndexedDB, LocalStorage, Sync	Voice transcripts, care sessio

1.2 Core Technology Stack

* Frontend Languages: Pure Vanilla ECMAScript 2022+ (ES6 Modules & Classic Scripts), HTML5 Canvas & WebGL, CSS3 Custom Properties (Design Tokens).
* Backend / Cloud Tier: Node.js / Express proxy endpoint support (/api/chat) for secure API key injection and streaming Server-Sent Events (SSE). Pure zero-backend fallback is built into the core client.
* 3D Graphic Rendering: Three.js r128 with custom GLSL simplex noise shaders for procedural organic wave deformation.
* Audio & Speech Services: Web Speech API (window.SpeechRecognition / window.webkitSpeechRecognition) for speech-to-text; window.speechSynthesis with dynamic rate/pitch modulation for text-to-speech.
* Data Persistence: IndexedDB (’caretrack_voice_ai_v1’) for permanent local voice turn transcripts; browser localStorage for state, user profiles, prescriptions, and mood logs.

2. Frontend Engineering & Design System

The frontend application is intentionally engineered with zero heavy framework bloat (no React, Angular, or Vue runtime dependencies). This guarantees instant load times, sub-10ms render latency, and smooth 60fps animations across both desktop and low-end mobile devices.

2.1 Application Shell & Mounting Lifecycle

When index.html is loaded by the browser, the runtime initializes in an exact chronological sequence:

* 1. CSS Assets Load: main.css -> components.css -> views.css -> ai-assistant.css -> auth.css.
* 2. Core Runtime: js/storage.js establishes the initial state tree (State.prescriptions, State.occurrences, State.profile, State.moodHistory).
* 3. Utility Libraries: js/utils.js exposes DOM helpers, icon renderer ic(), formatting utilities, and date arithmetic.
* 4. Views & Wellbeing: js/views.js and js/wellbeing.js register route listeners (home, schedule, tracker, wellbeing, profile).
* 5. AI Chat & Voice Modules: js/ai-engine.js and js/voice-mode.js mount floating action buttons (#ai-fab, #voice-fab) and initialize IndexedDB stores.
* 6. Watchdog & Boot: js/app.js triggers initApp() verifying authentication status, rendering the active route into #root, and starting real-time schedule alarms.

2.2 Responsive CSS Design System & Layout Orchestration

The design system in css/ai-assistant.css is structured with modern CSS Custom Properties, fluid typography, and strict boundary containment rules.

```
/* Desktop Floating AI Drawer */
.ai-panel.desktop {
  position: fixed; right: 24px; bottom: 96px;
  width: 440px; max-width: calc(100vw - 32px);
  height: min(630px, calc(100vh - 116px));
  border-radius: 20px; box-shadow: 0 20px 50px -10px rgba(15,23,42,0.25);
  overflow: hidden !important; contain: paint;
}

/* Mobile 100dvh Drawer Overlay */
@media (max-width: 640px) {
  .ai-panel, .ai-panel.desktop, .ai-panel.mobile {
    right: 0 !important; bottom: 0 !important; top: 0 !important; left: 0 !imp
    width: 100vw !important; height: 100dvh !important; border-radius: 0 !impo
  }
}
```

CSS Flexbox Minimum Width Gotcha Resolved

In CSS Flexbox specification, flex items have 'min-width: auto' by default. Without 'min-width: 0 !important' on .ai-msg, .msg-body, and .bubble, preformatted code, URLs, or tables expand the flex item beyond 100% of the popup. Adding explicit min-width: 0 and word-break: break-word completely eliminates bubble overflow.

3. The Sahaara AI Chat Engine (‘js/ai-engine.js‘)

The AI engine provides conversational guidance for daily medication routines, clinical instructions, habit adherence, and mental wellness. It features a dual-route architecture that balances intelligent cloud LLMs with deterministic client-side medical intelligence.

3.1 Dual-Core Routing & Request Lifecycle

When a user submits a query (via text input or speech recognition), the request follows a strict five-stage pipeline:

* Stage 1 (Emergency Interception): detectEmergency() screens for acute symptoms, suicidal ideation, or red-flag medical emergencies. If detected, normal routing halts instantly and emergency cards trigger.
* Stage 2 (Local Clinical Intelligence): sahaaraRoute() checks whether the query matches care plan inquiries, schedule questions, adherence stats, or relaxation exercises. Deterministic answers execute within 2ms.
* Stage 3 (Cloud LLM Proxy Request): If the query requires generative natural language, callLLM() sends a POST to AI_CONFIG.LLM_ENDPOINT (/api/chat).
* Stage 4 (Automatic Offline Fallback): If the endpoint returns 404, network error, or timeout, the engine sets AI.backendProxyUnavailable = true and falls back to sahaaraRoute() without alarming red toasts.
* Stage 5 (Streaming & Output Guard): guardSahaaraOutput() validates the response against clinical medication safety rules before rendering in the chat bubble.

3.2 Safety Guardrails & Medication Refusal Policy

Sahaara AI incorporates strict safety boundaries in adherence to clinical digital health guidelines:

* Dosing Alteration Defense: Questions asking whether to double, skip, stop, or change prescription dosages are systematically intercepted. The engine explicitly refuses to alter prescribed regimens and directs the patient to their clinician.
* Context Retention: buildLLMMessages() maintains clinical context, active prescriptions, and patient name while preserving system safety prompts.
* Markdown Lite Streaming Parser: renderMarkdownLite() securely sanitizes HTML, converts markdown bold/italic/quotes/lists/code blocks, and prevents XSS injection via esc().

4. Voice AI Mode & 3D WebGL Orb (‘js/voice-mode.js‘)

Voice Conversation Mode (#voice-fullscreen) delivers a full-screen, hands-free conversational experience powered by an organic 3D WebGL Orb, Web Speech recognition, and the Clinical Voice Engine.

4.1 Three.js 3D Procedural Shader Orb

The visual orb is constructed using Three.js WebGLRenderer rendering an IcosahedronGeometry (detail level 64) driven by custom GLSL vertex and fragment shaders:

```
// Custom GLSL Simplex 3D Noise Vertex Shader Snippet
vec3 v0 = position;
float noise = snoise(v0 * uNoiseFrequency + uTime * uNoiseSpeed);
vec3 newPos = v0 + normal * (noise * uDisplacementScale);
gl_Position = projectionMatrix * modelViewMatrix * vec4(newPos, 1.0);
```

The orb dynamically reacts to conversation state transitions:

* Idle State: Calming teal glow, gentle breathing pulsation (0.2x amplitude, slow rotation).
* Listening State: Dynamic audio reactivity driven by microphone volume pulses; expanded displacement waves.
* Thinking State: Amber hue with rotational acceleration indicating background inference.
* Speaking State: Purple/cyan pulsing modulated by TTS speech boundary events and utterance character lengths.

4.2 The Sahaara Voice Engine (Fetch-Decode-Execute Architecture)

Voice commands require sub-second response times. Sahaara Voice Engine bypasses slow server roundtrips using a dedicated 3-stage local pipeline:

* 1. Fetch Stage: SahaaraVoiceEngine.fetch() aggregates live state: today’s schedule, pending doses, completed tasks, weekly adherence %, and recent mood scores.
* 2. Decode Stage: SahaaraVoiceEngine.decode() analyzes natural language intents using regex token heuristics (e.g., COMPLETE, SCHEDULE_NEXT, ADHERENCE, BREATHING, EMERGENCY_SOS, MOOD_LOG).
* 3. Execute Stage: SahaaraVoiceEngine.execute() mutates state (marks dose complete, saves mood), triggers 3D orb ripple animations, and synthesizes localized speech feedback in English, Hindi, or Hinglish.

5. Clinical Data Modeling & Persistence Architecture

Sahaara AI manages complex self-care plans comprising prescription medications, hydration, breathing exercises, physical activity, and mood logging.

5.1 State Data Model Schema

The application maintains a single source of truth (State) synchronized to browser storage:

```
State = {
  prescriptions: [
    { id, title, category, dosage, instructions, times: ['08:00','20:00'], act
  ],
  occurrences: {
    '2026-10-31::rx_101::08:00': { status: 'completed', completedAt: '2026-10-
  },
  profile: { name, age, primaryCondition, emergencyContact: { name, phone } },
  moodHistory: [ { id, date, score: 1-5, note, loggedAt } ]
};
```

5.2 Adherence Mathematics & Streak Tracking

Clinical adherence is calculated in real-time across rolling date ranges using calcDetailedAdherence():

* Formula: Adherence % = (Completed Occurrences / Total Scheduled Occurrences) * 100.
* Streak Calculation: Consecutive days where adherence >= 80% increment the streak counter. Missed doses reset the active streak, encouraging daily patient engagement.

5.3 IndexedDB Storage Architecture (‘caretrack_voice_ai_v1‘)

Because voice transcripts can grow into thousands of turns, localStorage would quickly hit its 5MB quota. Sahaara AI utilizes IndexedDB for persistent voice turn storage:

* Database: ’caretrack_voice_ai_v1’ (Version 1).
* Object Store: ’turns’ with primary key ’id’ (’voice::sessionId::messageId’).
* Indexes: ’sessionId’ for instant conversation retrieval; ’createdAt’ for timeline sorting.
* Retention Policy: LRU eviction prunes turns exceeding 500 records per session to maintain high performance.

6. Emergency SOS & Crisis Response Protocols

In patient-facing healthcare applications, immediate crisis detection is safety-critical. Sahaara AI embeds priority interruption protocols across both text chat and voice bot.

6.1 Crisis Interception Heuristics

detectEmergency(text) evaluates text against clinical safety lexicons across English, Hindi, and regional dialects:

* Suicide & Self-Harm: Patterns detecting intent to self-harm, hopelessness, or overdose immediately trigger the Tele-MANAS (14416), KIRAN (1800-599-0019), and Vandrevala Foundation crisis hotlines.
* Acute Medical Symptoms: Severe chest pain, shortness of breath, slurred speech, acute bleeding, or sudden paralysis trigger National Emergency Services (112 / 108).

6.2 Barge-In & Stream Abortion Protocol

When an emergency is detected while the AI is speaking or streaming a response:

* 1. Priority Barge-In: Active speech synthesis and audio buffers cancel within 1ms.
* 2. Fetch Abortion: AbortController signals the active LLM stream to terminate immediately.
* 3. UI Transition: The voice orb shifts to urgent red (#dc2626) and renders .voice-emergency-card.
* 4. Direct Calling Action: Tap-to-call buttons (<a href='tel:112'>) provide zero-friction emergency dialing.

7. Extensibility, Integration & Production Guide

Sahaara AI is structured for easy deployment, white-label adaptation, and hospital EHR integration:

* Deployment Options: Can be hosted statically on GitHub Pages, AWS S3 / CloudFront, Netlify, Vercel, or inside hospital intranet web servers.
* Connecting Live LLM Endpoints: Set AI_CONFIG.LLM_ENDPOINT to your backend gateway (e.g. Node.js Express, Python FastAPI, or Vertex AI Proxy). The client automatically verifies connectivity.
* FHIR / HL7 Standards: The prescription and occurrence data model maps directly to HL7 FHIR MedicationRequest and MedicationStatement resources.
* Security & Privacy: All voice recognition and local data processing remain strictly on the patient's device. No raw audio is recorded or stored on remote servers.